Visual Comput (2006) 22: 315–323
DOI 10.1007/s00371-006-0012-8

ORIGINAL ARTICLE

Changbo Wang
Zhangye Wang
Tian Xia
Qunsheng Peng

Real-time snowing simulation

Published online: 20 April 2006

C. Wang (✉) · Z. Wang · T. Xia · Q. Peng
State Key Laboratory of CAD & CG,
Zhejiang University, P.R. China

C. Wang
Department of Building Engineering,
Tongji University, Shanghai, P.R. China

Abstract A snowing scene has a unique fascination for people due to its incomparable beauty. However, little work has been presented on the real-time generation of a dynamic snowing scene, partially due to the difficulty that the simulation of a dynamic snowing process involves the complex modeling of the wind field and the interaction between wind and snow. In this paper, by fully considering the physical characteristics of wind and snow, we construct a three-dimensional wind field based on the discrete form of the Boltzmann equation. According to the interaction laws between wind and snow, we simulate the falling of snow, deposition and erosion in 3D space. Experimental results show that realistic wind-driven snow scenes under different speeds of wind with different amounts of snowfall can be rendered in real-time.

Keywords Dynamic snowing scene · Real-time simulation · Boltzmann equation · 3D wind field

1 Introduction

Real-time modeling and rendering of photorealistic natural phenomena, such as the simulation of terrain, trees, flows, clouds, smoke, fog, fire, etc., has been a challenge in computer graphics. Compared with other natural phenomena, the snowing scene has a unique fascination for people due to its incomparable beauty. The simulation of a snowing scene is, therefore, indispensable in areas such as computer animation, PC games, special movie effects, and so on.

Since snowflakes are very light, their falling is subject to a wind field and to a vortex. The deposition of snow is also affected by many factors. No wonder, it is a very difficult task to model and render the snowing scene in real-time.

A lot of effort has been put into simulating a snowy scene. Reeves et al. [15] first simulated falling snow with a particle system. However, the original simulation results are less realistic due to the incorporation of an over-simple set of rules. Nishita et al. [13] adopted metaballs to model and render the scene of snow accumulation and to simulate light scattering through the snow. Nevertheless, their algorithm is not suitable for real-time simulation. Paul Fearing [6] described a complex stability model to construct fallen snow by geometrical surfaces, and generated the most realistic falling snow scene to date. However, due to the huge scene data and the time-consuming modeling process, its rendering speed is quite low. Chen et al. [4] employed the techniques of multi-mapping and displacement mapping to produce a realistic snowy scene. Ohlsson et al. [14] adopted a depth image method to calculate the amount of fallen snow on each mesh of the terrain and perform occlusion culling, thus they could render the scene of falling snow quite fast. However, this approach demands a great amount of pre-processing time. Felman et al. [7] calculated the deposition of snow by solving the 3D Navier–Stokes equations set. Haglund et al. [8] modeled snowflakes with triangle meshes and simulated the process of snow deposition by computing a matrix that keeps record of the thickness of deposition at each sam-

pling point. Thomas et al. [17] numerically simulated the effect of snow deposition around buildings.

While most previous works focus on the simulation of fallen snow, little work has been done on the snowing scene and on the interaction between wind and snowflakes. Moreover, their research is mainly limited to numerical simulations. For example, Sumner et al. [16] computed the shape of fallen snow on the ground due to different external pressure at the height of the current surface, and this work focused on the simulation of physical aspects. Masselot et al. [11] introduced the interaction laws between wind and snow. However, their approach only accounted for the 2D case. Corripio et al. [4] proposed a simple numerical simulation of the wind influence on snow. Langer et al. [10] used the method of inverse 3D Fourier transformation to deal with scene images and reconstructed the snowing scenes by an image series. Apparently this method is in fact an image processing technique. Since the wind-snow interaction was neglected, the rendered scene was not quite realistic. For wind field simulation, Jakub et al. [9] proposed a model based on aerodynamics and Wei et al. [18] simulated a feather flying in the wind.

In this paper, by fully considering the physical characteristics of wind and snow, we construct a three-dimensional wind model adhering to the classical Boltzmann equation. Then, based on the interactive laws between wind and snowflakes, we model the flight of snowflakes in air, and simulate the dynamic deposition and erosion of snow on the ground. By proper simplification, realistic wind-driven snow scenes under different speeds of wind with different amounts of snowfall can be rendered in real-time.

The rest of this paper is organized as follows. In the next section, we describe the construction of the wind field model based on the classical Boltzmann equation. In Sect. 3, we discuss how to model the interaction between wind and snow. A realistic rendering algorithm for a wind-driven snow scene is proposed in Sect. 4. The simulation results of wind-driven snow scenes in real-time under different conditions are presented in Sect. 5. Conclusions and highlights for future work are presented in the last section.

2 Wind field model based on the Boltzmann equation

When simulating the wind-snow scene, it is essential to establish the model of the wind field.

Generally, there are two ways to study the aerodynamic characteristics of the wind field. One is based on the assumption that the fluid is continuous in both time and space. The most well-known model of this type is the Navier–Stokes differential equations set. However, as the model is a nonlinear equation set, it is quite difficult to obtain an analytical solution in many circumstances. It is hence more difficult to dynamically simulate a snowing scene accounting for interaction among vortex, wind and snow.

Another way is to employ the microscopic mechanism based on statistical physics. The Boltzmann equation is a typical model of this type, which studies the laws of particle clusters' distribution instead of the motion of an individual particle. The Boltzmann equation conveys deeper physical meaning than the Navier-Stokes equation, which are grounded on the assumption of a continuous medium [2].

However, the Boltzmann equation is also difficult to solve; thus the idea of discrete kinetics is introduced here. Considering that the flow field is the macroscopic effect of moving particles, time and space are fully discretized. The wind field is sampled at many grids, and the moving state of air particles at the grid nodes is described by distribution functions. Let those particles move along the grid lines, and collide with each other at the grid according to certain rules. The evolution of the distribution function reflects the motion law of wind macrocosmically.

According to this idea, the Boltzmann equation with discrete velocities without outer force items can be expressed as:

$$\frac{\partial f_i}{\partial t} + e_i \cdot \nabla f_i = \frac{1}{\tau}(f_i^{eq} - f_i), \tag{1}$$

where the right-hand term is the BGK collision term, among which f_i is the current particle density, f_i^{eq} is the local equilibrium distribution function, and τ is the relaxation time that represents the inclination time from the non-balance status to the equilibrium state.

According to Boltzmann's H-Theorem [2], a system in non-equilibrium state tends to its equilibrium state with maximal probability. Because of the motion and collision of particles, their distribution functions transport from one grid to another in discrete time steps and incline to a local equilibrium state. Furthermore, it obeys the conservation law of mass and momentum during this process.

If we adopt 2D grids as shown in Fig. 1, i.e. it is possible for each particle to move along eight directions, the discrete form of the Boltzmann equation can be expressed

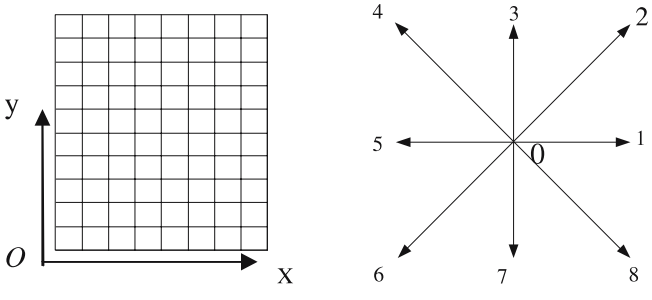


Fig. 1. 2D grid subdivision and discrete velocity vectors

as [11]:

$$f_i(t+\Delta t, x+\boldsymbol{c}_i\Delta t) - f_i(t,x) = \frac{1}{\tau}(f_i^{eq}(x,t) - f_i(x,t))$$
$$i = 0, 1, 2, \cdots, 8, \tag{2}$$

where f_i is the density of particles with velocity $\boldsymbol{c}_i$ at the point x and at time t, Δt is the time interval during which particles travel along a grid spacing.

At each grid node, the density and macroscopic velocity of air particles can be expressed by the distribution function as follows:

$$\begin{cases} \rho(x,t) = \sum_{i=0}^{8} f_i(x,t) \\ u(x,t) = \frac{1}{\rho}\sum_{i=0}^{8} \boldsymbol{c}_i f_i(x,t) \end{cases}, \tag{3}$$

Considering the symmetry of the grids' shape and the gas transport theory, which includes that air particles obey the conservation laws of mass and momentum during their motion, we can easily determine the local equilibrium distribution function f_i^{eq}, then wind field is simulated by discretizing the Boltzmann equation.

It is not difficult to prove that from the above-derived microscopic kinematics equations, the following macroscopic kinematics equation (deduction details are omitted) can be recovered:

$$\begin{cases} \frac{\partial \rho}{\partial t} + \frac{\partial \rho u}{\partial x} + \frac{\partial \rho v}{\partial y} = 0 \\ \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -g\frac{\partial \rho}{\partial x} + fv \\ \frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -g\frac{\partial \rho}{\partial y} + fu \end{cases}. \tag{4}$$

Equation 4 is the classic aerodynamics continuity Navier–Stokes equation, and that the Navier–Stokes equation is actually a low-order approximation to the Boltzmann equation [3]. The discretization of the Boltzmann equation simplifies the calculation and makes it possible to simulate the wind field in real-time.

3 Modeling of wind-driven snowflakes

3.1 Wind modeling

The modeling of a wind-driven snow scene can be divided into two main parts: one is the modeling of wind field, and the other is the modeling of snowflakes and their interaction with wind.

3.1.1 Three-dimensional wind field model

We extend the above wind field model to three dimensions. The three-dimensional space is sampled uniformly and the total number of grid nodes is $N_x \times N_x \times N_z$. At each grid node, the wind distribution is represented by $F_i(r,t)$, which is the fluid density moving along the i-th direction, shown by $\boldsymbol{c}_i$, where r is the current grid and t represents time. We adopt 15 discrete directions instead of 27 to reduce the amount of computations [18]. The model is shown in Fig. 2.

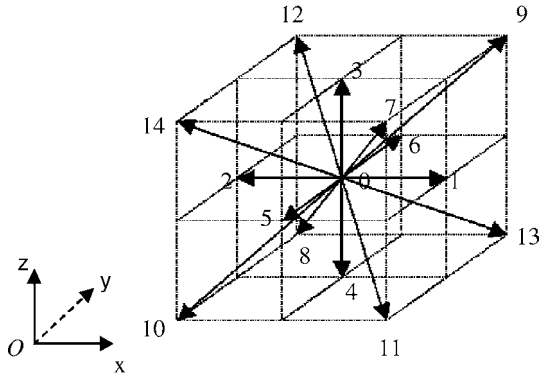


Fig. 2. Discrete velocity of a three-dimensional wind field

There are fifteen directions represented from $i = 0$ to $i = 14$, and the values of the discrete velocities $\boldsymbol{c}_i$ can be expressed by the following formula [12]:

$$\boldsymbol{c}_i = \begin{cases} (0,0,0) & : \quad i = 0; \\ (\pm 1, 0, 0)c, (0, \pm 1, 0)c, (0, 0, \pm 1)c & : \\ & i = 1, 2, \cdots, 6; \\ (\pm 1, \pm 1, \pm 1)c & : \\ & i = 7, 8, \cdots, 14; \end{cases}. \tag{5}$$

At each grid node, the density of the wind fluid ρ and the speed u are, respectively:

$$\rho = \sum_{i=0}^{14} F_i, \quad u = \frac{1}{\rho}\sum_{i=0}^{14} F_i \boldsymbol{c}_i. \tag{6}$$

Similar to Eq. 2, the three-dimensional discrete wind field can be expressed as:

$$F_i(r+\boldsymbol{c}_i, t+\Delta t) = F_i(r,t) + \frac{1}{\xi}(F_i^{eq}(u(r,t), \rho(r,t)) - F_i(r,t)), \tag{7}$$

where ξ is the relaxation time according to the local velocity gradients, which is related to the fluid viscosity. The value of ξ can be 0.5 to reach a very large Reynolds number. The local equilibrium distribution $F_i^{eq}(u, \rho)$ is:

$$F_i^{eq}(u,\rho) = \omega_i \rho \left[1 + \frac{\boldsymbol{c}_{i\alpha}u_\alpha}{c_s^2} + \left(\frac{\boldsymbol{c}_{i\alpha}u_\alpha}{c_s^2}\right)^2 - \frac{u_\alpha \cdot u_\alpha}{2c_s^2}\right],$$
$$i = 0, 1, 2, \cdots, 14, \tag{8}$$

where $\boldsymbol{c}_{i\alpha}$ represents the coordinate value α of the discrete velocities $\boldsymbol{c}_i$ and $c_s^2 = 1/3$, and the values of the weighing factor ω_i is:

$$\omega_i = \begin{cases} 2/9, & i = 0; \quad \textit{rest particles} \\ 1/9, & i = 1, 2, \cdots, 6; \quad \textit{group I} \\ 1/72, & i = 7, 8, \cdots, 14; \quad \textit{group II} \end{cases} \tag{9}$$

3.1.2 Boundary conditions

The movement of wind is often influenced by the shape of terrain, buildings and landscape (snow fences), and so on. So we set boundary conditions as follows [5].

- **The upper boundary:** is sky here, at which the speed along the vertical direction is set to 0, and along the horizontal direction is set to the desired numbers. This would be near to the so-called bouncing forward condition.
- **The side boundaries:** we prefer to introduce a constant profile $u_{hor} \leftarrow u_\infty$ and allow a long enough distance in front of the wind tunnel to let the profile reach its steady state by itself.
- **The ground:** includes the terrain, buildings, and some man-made snow fences. All grid nodes are solid here, i.e. the speed of every particle of this type is 0. Their common condition is the bouncing back condition. On a solid site, we reverse all "moving"F_i. In the grid node, we apply:

$$F_i(r_{solid}, t) \leftarrow F_{i-(-1)^i \mod 2}(r_{solid}, t).$$

3.2 Snowing modeling

The snowflakes mainly act in wind under three kinds of forces: gravity, force of wind and the collision force among the snowflakes. As the frequency of collision between snowflakes is quite small, we can omit the collision force among snowflakes. Gravity is always downward and its value can be expressed inmg. The influence of wind on snow is significant, so we emphasize the interaction between wind and snow, and neglect the collision between snowflakes. The wind-snow interaction can be expressed as follows: (1) dancing of the snowflakes in the wind during snowing, (2) salutation of the snowflakes within the height of 30 centimeters to the ground, and (3) erosion of the snow on the ground [1].

According to the regularity of wind-snow interaction, we establish the interactive transition models of the falling of snow, deposition and erosion. Then, we can realistically simulate the scene of snow flying, circumvolving and accumulation in the wind. We will discuss these in detail separately.

(1) Flying in the wind. If we neglect the inertia effects, a snowflake will move subjected to the velocity field of wind $u(r, t)$ and gravity. Let the vertical falling speed of a snowflake be $-u_g$, where the minus sign indicates the downward direction. Thus, a snowflake on the site $r = (r_x, r_y, r_z)$ should go, after Δt time, to $r' = (r'_x, r'_y, r'_z)$, where $r'(t + \Delta t) = r(t) + (u - u_g)\Delta t$, that is:

$$r'_x = r_x + u_x \Delta t, \quad r'_y = r_y + u_y \Delta t, \quad r'_z = r_z + (u_z - u_g)\Delta t. \tag{10}$$

Note that the velocity of the wind field at each grid node can be determined by the method presented in Sect. 3.1.

If r' does not coincide with any grid node, we assume that the neighbouring 3D grids of the r' are $P_i (i = 0, 1, \cdots, 7)$ and the velocity of the wind field at P_i is $V_i (i = 0, 1, \cdots, 7)$. Then we determine the velocity of the wind field at r', i.e. $V_{r'}$, by linear interpolation.

Specifically, suppose the coordinates of P_i are (x_i, y_i, z_i) $(i = 0, 1, \cdots, 7)$, let $\Delta x = \frac{(x - x_0)}{\Delta \Gamma_x}$, $\Delta y = \frac{(y - y_0)}{\Delta \Gamma_y}$, $\Delta z = \frac{(z - z_0)}{\Delta \Gamma_z}$, where $\Delta \Gamma_x, \Delta \Gamma_y, \Delta \Gamma_z$ represent, respectively, the span of adjacent grids along x, y, z directions. We have:

$$\begin{aligned} V_{r'} = {} & (1 - \Delta x)(1 - \Delta y)(1 - \Delta z) \cdot V_0 \\ & + (1 - \Delta x)(1 - \Delta y) \cdot \Delta z \cdot V_1 \\ & + (1 - \Delta x)(1 - \Delta z) \cdot \Delta y \cdot V_2 \\ & + (1 - \Delta y)(1 - \Delta z) \cdot \Delta x \cdot V_3 + (1 - \Delta x)\Delta y \cdot \Delta z \cdot V_4 \\ & + (1 - \Delta y) \cdot \Delta x \cdot \Delta z \cdot V_5 + (1 - \Delta z) \cdot \Delta x \cdot \Delta y \cdot V_6 \\ & + \Delta x \cdot \Delta y \cdot \Delta z \cdot V_7. \end{aligned} \tag{11}$$

(2) Snow deposition. Deposition plays an important role in the simulation of snowfall. After a snowflake falls and reaches the ground, it reaches a stable site and gets solidified.

To simulate the dynamic snow deposition, we first need to generate the height field $h(x, y)$ of the ground. This can be achieved by applying a z-buffer algorithm to the 3D ground model taking the the sky as being infinite as the viewpoint. Here the ground includes the terrain, buildings, some man-made snow fences, etc.

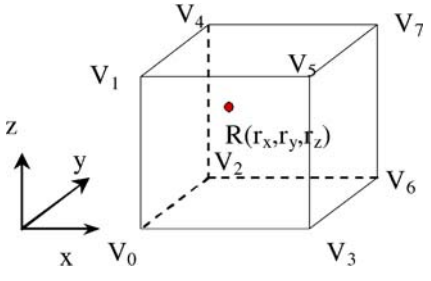


Fig. 3. The velocity of snowflakes in a wind field

The height field is, therefore, represented in a discrete form $h(x_i, y_j)(i = 0, 1, \cdots, m; j = 0, 1, \cdots, n)$. After a snowflake falls to the height $h(x_i, y_j)$, it reaches the site (x_i, y_j) and accumulates there. After plenty of snowflakes deposit at the same site, the height of this site is increased and updated.

We adopt a multi-particle model here, i.e. only when one site receives a sufficient amount of snowflakes greater than a given threshold, will the locally deposited snow become solidified and the height at this site will be increased by 1, i.e. $h(x_i, y_j) = h(x_i, y_j) + 1$. We associate each site with a snowflake counter and use texture with different gray levels as well as different alpha values to reflect the amount of snow.

In our current implementation, the accumulation of bridge snow blocks and the snow deposition below occlusion objects are not accounted for. These effects can be achieved by setting a multi-layer height field.

(3) Erosion. Deposited snowflakes may be eroded under some conditions. If the wind is strong enough (greater than a given threshold), deposited snow particles will be pulled forwards and move along the ground by wind bursts resulting from the interaction of slow and fast bands on the surface. On the one hand, deposited snowflakes are pulled up by strong wind. On the other hand, flying snow particles reduce the viscosity of the wind and slow down the erosion process. The erosion rate of snow seems to be related to the wind speed over the solid site, the concentration of snow being transported, the saturation concentration and the efficiency of the transport [10]. In our model, we express these mechanisms in a simple way. The erosion rate can be expressed as:

$$e_i = \alpha_{snow}(u_{si} - u_{thres}). \tag{12}$$

Here, u_{thres} is the threshold of the wind velocity beyond which erosion occurs, and u_{si} is the current wind velocity over the ground site (x_i, y_j). The efficiency of the transport α_{snow}, has some relationship to snow density, weather and zone, etc. When it is snowing, the value of this coefficient can be: $\alpha_{snow} = 0.1533$.

If $u_{si} > u_{thres}$, we set $h(x_i, y_j) = h(x_i, y_j) - 1$, $u_{si} = u_{si} - \Delta u$. When $h(x_i, y_j)$ changes to its initial value, this site takes the ground color. Eroded snowflakes will fly in wind or deposit and erode again like other snowflakes.

4 Rendering of a snowing scene

4.1 The snowflake model

There are many kinds of beautiful shapes of snowflakes in the real world as shown in Fig. 4. Since our system must generate a great number of snowflakes, we introduce the following two models of snowflakes for real-time simulation.

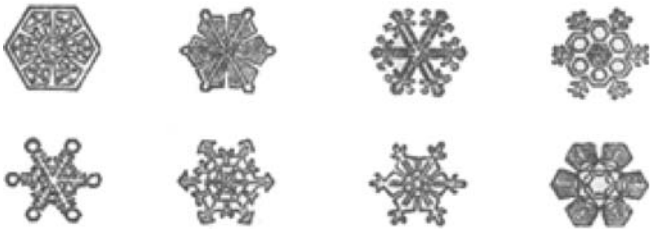


Fig. 4. Various shapes of snowflakes

(1) The shape of the sphere. The snow particle locates at the center of the sphere. The grayscale of the sphere changes from the center to the boundary according to the Gauss distribution. The radius of the sphere is determined by a certain probability. Supposing that the radius of the sphere in texture space is R, the radius of the snow in the scene can be determined by $R' = kR$. Here k is a random number whose value is (0, 1]. R' is usually less than 5 mm.

The diameter of the snowflake is in relation to with the climate and can be set interactively. It is shown that the diameter of snowflakes in a location with a higher latitude is larger than that in a location with lower latitude if other conditions are the same.

(2) The more realistic shape of a plum-blossom. Taking the shapes of the plum-blossom shown in Fig. 4 as texture, we render each snowflake using the billboard technique. Different models can be adopted according to different scenes.

4.2 Establishing the wind field

Due to the complex mechanism of wind transportation, it is unlikely that wind keeps a constant speed all the time. Suppose that the wind velocity is composed of two parts: the average velocity $\bar{V}_w$ and a random component $\tilde{V}_w$, i.e. $V_w = \bar{V}_w + \tilde{V}_w$. The value of the average velocity can be obtained from meteoric data. Moreover, the value of the random velocity can be determined by following formula:

$$\tilde{V}_w = \mu \bar{V}_w \sin\alpha, \quad \alpha \in [0, \pi], \tag{13}$$

where μ is a random coefficient whose value changes according to the wind velocity. We set $\mu = n/20$, where n is the scale of the wind force; α is the phase angle whose value can be a random number ranging from 0 to 180. This step improves the realistic effect of our model.

The wind field is established by changing the wind particle density along different directions at grid nodes. Two major approaches to constructing a 3D wind field are adapted.

(1) Introducing the wind velocity V_w to specified boundary nodes simultaneously. The strong wind field

can be established in the way of a flood fill algorithm. Specifically, for each grid node to be filled, we add the respective wind particle density transferred from its neighboring grid along each direction with different weighting factors. Suppose the wind particle density of a selected node is ρ and the direction of the wind field is $\bar{C}_w$. For each direction i, the change of wind particle density is $\Delta F_i, i = 0, 1, \cdots, 14$ (see Fig. 2). ΔF_i is then expressed as:

$$\Delta F_i = \lambda_i \varepsilon_i \rho V_w, \tag{14}$$

$$\text{where } \lambda_i = \begin{cases} 1/4, & \Delta c_i = 0 \\ 1/16, & \Delta c_i \in (0, \pi/2), \\ 0, & \Delta c_i = \pi/2 \end{cases}$$

$$\varepsilon_i = \begin{cases} 1 & \Delta c_i \in [0, \pi/2] \\ -1 & \Delta c_i \in (\pi/2, \pi] \end{cases}.$$

Here Δc_i is the angle between $\boldsymbol{c}_i$ and $\boldsymbol{C}_w$, $\boldsymbol{c}_i$ represents one of fifteen discrete directions as shown in Fig. 2. Following Eq. 3, which calculates the mass and momentum at each grid node, the increase in the velocity at the node is $\Delta u = \frac{1}{\rho} \sum_{i=0}^{14} \boldsymbol{c}_i \Delta F_i = V_w$, and the increase in the density is $\Delta \rho = \sum_{i=0}^{14} \Delta F_i = 0$. That is, the wind speed V_w is added to the specified nodes while the conservation of mass is locally satisfied.

(2) Introducing an acceleration constant a. After time t, the wind speed accumulates to V_w, $V_w = at$. This approach is suitable for establishing a light wind field. The propagation process is similar to that of the first approach.

4.3 Real-time rendering of a snowing scene

The initialization of our rendering process includes the following steps. (1) Divide the 3D space into $N_x \times N_y \times N_z$ grids and set the coordinate system as shown in Fig. 2. (2) Initialize the wind field. We set the initial momentum at each grid to zero and set the initial density according to the rule of local equilibrium, that is, the particle density at each direction is $\omega_i \rho_0$; here ω_i is determined by Eq. 9 and ρ_0 is the initial density of the wind fluid. (3) Distribute the snow particles. We periodically add snow particles to the system from the upper boundary and the left boundary according to a certain probability. (4) Set the boundary conditions according to the method as described in Sect. 3.1.2. (5) Initialize the height field $h(x, y)$ of three-dimensional ground.

The pseudo-code of whole process is:

Initialization:
{ *divide the 3D space into grids*
initialize wind particles
distribute snow particles
initialize the height field of the ground
set the boundary conditions of the wind field
}
Snowing simulation:
For each time step

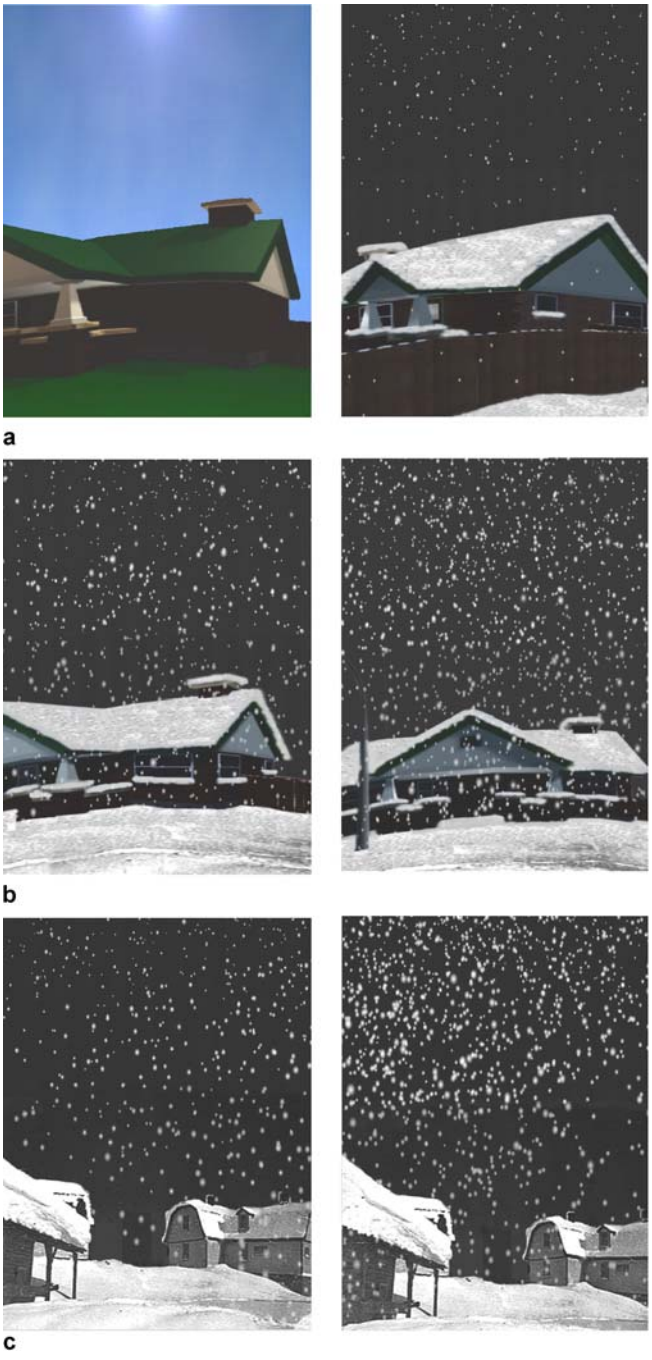


Fig. 5a–c. Simulation results of different snowing scenes **a** Sunny day and light snowing scene. **b** Moderate and heavy snowing scene. **c** Snowing in different wind conditions (three-level rightward and five-level leftward wind)

```
{   model the interactions between wind
        and snowflakes
    update the state of the wind particles
    update the position of the snowflakes
    if (any snow particles touch the ground)
        deposit and erode
    render the snowing scene
}
```

During the simulation, we can change the strength and direction of the wind interactively and, therefore, render snowing scenes under different conditions. Besides the incorporation of environment mapping for rendering the snowing sky, we have also imported a 3D ground scene to make the whole scene more realistic. Other skills are also used here to speed up the rendering process, including the employment of the display list of OpenGL to render a large number of snowflakes.

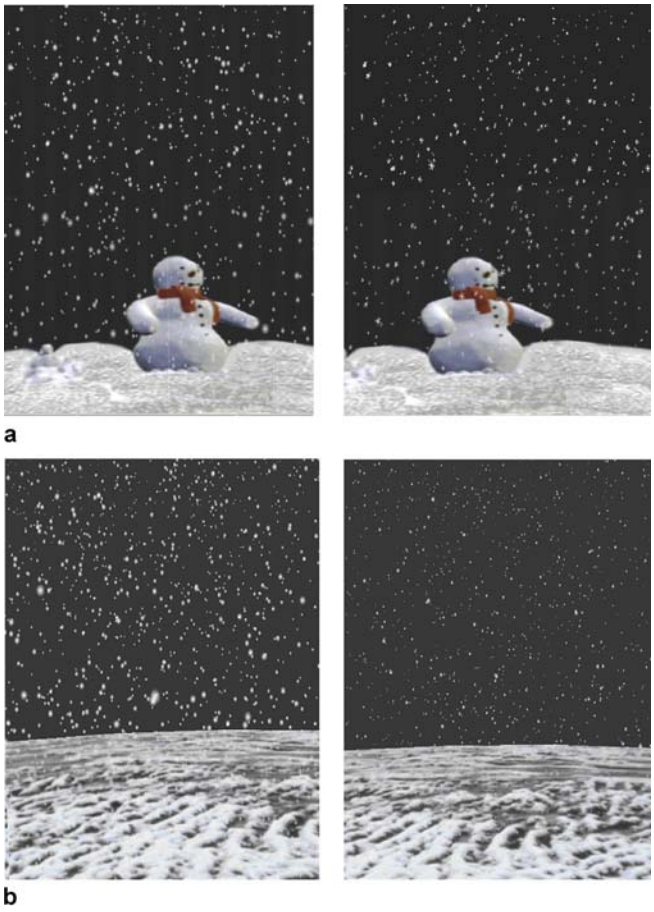


Fig. 6a,b. Snowflakes with different shapes and sizes **a** Sphere and plum-blossom snowflakes **b** Big and small snowflakes

5 Results

Applying the method proposed in this paper, we have rendered several kinds of wind-driven snow scenes (with a lattice of $150 \times 100 \times 60$, and an image resolution of 600×400) on a PC of Pentium IV 2.4 GHz, 2.0 GB memory at the State Key Laboratory of CAD & CG, Zhejiang University (see Figs. 5–9). The average rendering speed reaches 26 frames per second and thus meets our goal of real-time rendering.

The left-hand side image of Fig. 5(a) shows a scene on a sunny day. The right-hand side images of Figs. 5(a) and (b) represent the scenes with different snowing conditions. Fig. 5(c) is the simulation results of two snowing scenes under different wind conditions. Fig. 6 shows snowing scenes with snowflakes with different shapes and sizes. From these figures, we can see that our simulation results are quite satisfactory. Fig. 7 and Fig. 8 show the simulation results of snow flying, deposition and erosion at different time steps from the same viewpoint and from different viewpoints at the same time, respectively. We assume that a fourth-grade wind blows from the left of the house to the right. As shown in these figures, there is more snow deposited on the windward side of the roof than that of on the downwind side. Similarly, more snow has fallen to the left side of the house. This is due to the presence of the house, which causes a vortex that makes more snow deposit there. Fig. 9 shows the simulation result without wind, where snow deposition is of almost the same thickness on both sides of the house. These examples demonstrate the potentials of our model.

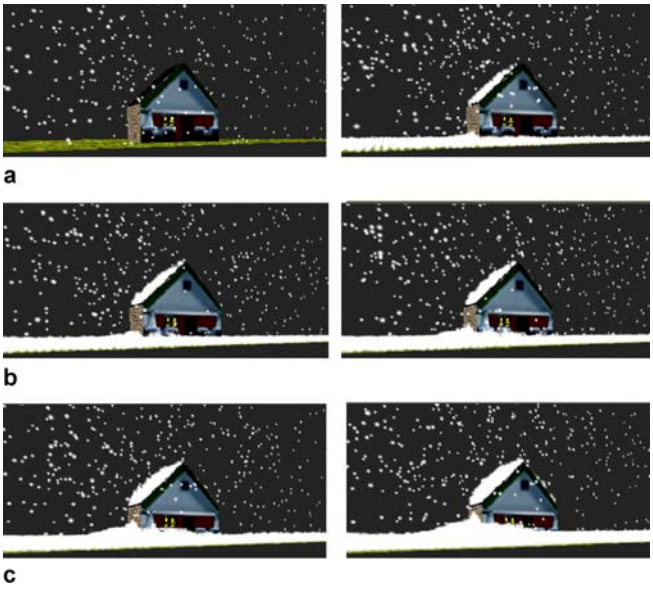


Fig. 7a–c. The same view at different times **a** At time $t = 0$ and $t = 1000$ **b** At time $t = 2000$ and $t = 3000$ **c** At time $t = 5000$ and $t = 7000$

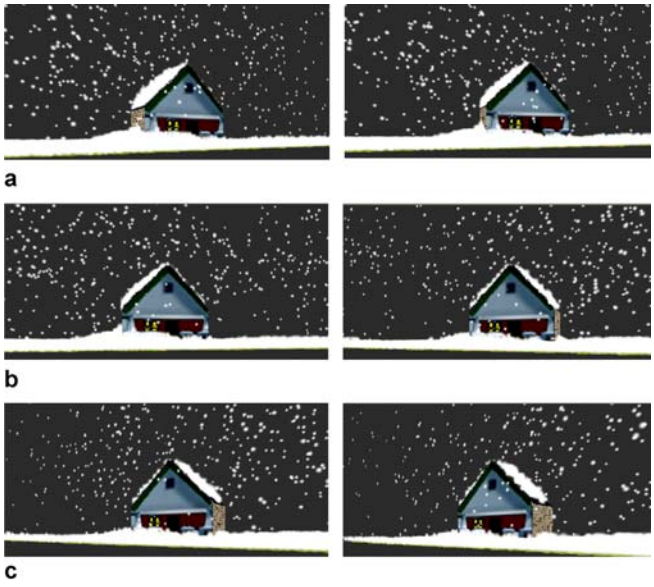


Fig. 8. Different viewpoints at the same time step

Fig. 9. Snow deposition without wind

6 Conclusions

We have proposed a real-time rendering method to simulate the wind-driven snow scene. By adopting the discrete form of the classical Boltzmann equation, we construct a 3D wind field model and simulate snowfall, deposition and erosion at each grid concerned in 3D space based on the interaction laws between wind and snow. Different wind-driven snow scenes are modeled under different conditions. Since the aerodynamic properties of the wind field and wind-snow interaction are taken into account, the simulation results of the snowing scene are very realistic. By introducing various simplification and acceleration techniques, we are able to render different wind-driven snow scenes in real-time.

Future research work includes: (1) optimizing the model of wind and snow interaction, including providing more intuitive interactive means; (2) simulating bridge snow blocks to enhance the reality of the snowing scenes; (3) applying GPU to accelerate the rendering process, etc.

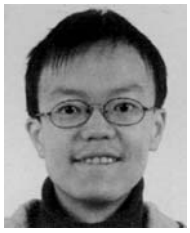
Acknowledgement This research was supported by the 973 program of China under grant no. 2002CB312101, the National Natural Science Foundation of China under grant no. 60033010 and a joint project supported by the National Natural Science Foundation of China and the Research Grant Council of Hong Kong.

References

1. Bang, B., Nielesen, A., Sundsbo, P.-A., Wiik, T.: Computer simulation of wind speed, wind pressure and snow accumulation around buildings (SNOW-SIM). Energ. Buildings **21**(3), 235–243 (1994)
2. Chapman, S., Cowling, T.-G.: The mathematical theory of non-uniform gases, 3rd edn. Cambridge University Press, Cambridge (1970)
3. Chen, S., Doolen, G.-D.: Lattice Boltzmann method for fluid flows. Annu. Rev. Fluid Mech. **30**(1), 329–364 (2001)
4. Chen, Y.-Y., Sun, H.-Q., Guo, B.-L., Wu, E.-H.: Modeling and rendering snowy natural scenery. Chinese J. Comput. **25**(9), 916–922 (2002)
5. Corripio, J.-G., Durand, Y., Guyomarch, G., Merindol, L., Lecorps, D., Puglises, P.: Modelling and monitoring snow redistribution by wind. Cold Reg. Sci. Technol. **39**(2/3), 93–104 (2004)
6. Fearing, P.: Computer modeling of fallen snow. In: K. Akeley, J. White (eds.) Proceedings of ACM SIGGRAPH '2000, pp. 37–46. ACM Press, New Orleans (2000)
7. Feldman, B.-E., O'Brien, J.-F.: Modeling the accumulation of wind-driven snow. Technical sketch. In: T. Appolloni (ed.) Proc. ACM SIGGRAPH 2002 Conference, Abstracts and Applications, TX, pp. 9–12. ACM Press, San Antonio (2002)
8. Haglund, H., Andersson, M., Anders, H.: Snow accumulation in real-time. In: M. Ollila (ed.) Proc. of SIGRAD'2002, pp. 11–15. Linkoping University Electronic Press, Norrkoping, Sweden, (2002)
9. Wejchert, J., Haumann, D.: Animation aerodynamics. Comput. Graphics **25**(4), 19–22 (1991)
10. Langer, M.-S., Zhang, L.-Q.: Rendering falling snow using an inverse Fourier transform. In: A.P. Rockwood, J. Hodgins (eds.) Proc. ACM SIGGRAPH'2003, pp. 58–64. ACM Press, San Diego, CA (2003)
11. Masselot, A., Chopard, B.: Lattice gas modeling of snow transport by wind. In: J. Dongarra, K. Madsen, J. Wasniewski (eds.) Proc. of Applied Parallel Computing, Computations in Physics, Chemistry and Engineering Science (PAPA), pp. 429–435. Springer, Lyngby, Denmark (1995)
12. Mei, R.-W., Wei, S.-Y., Yu, D.-Z., Luo, L.-S.: Lattice Boltzmann method for 3-D with curved boundary flows. J. Comput. Phys. **161**(2), 680–699 (2000)
13. Nishita, T., Iwasaki, H., Dobashi, Y., Nakamae, E.-A.: A modeling and rendering method for snow by using metaballs. Comput. Graph. Forum **16**(3), 357–364 (1997)
14. Ohlsson, P., Seipel, S.: Real-time rendering of accumulated snow. In: S. Seipel (ed.) SIGRAD 2004: Special theme – Environmental Visualization. Linkoping Electronic Conference Proceedings, pp. 25–32. Linkoping University Electronic Press, Gävle, Sweden (2004)
15. Reeves, W.-T., Ltd, L.: Particle systems – a technique for modeling a class of fuzzy objects. ACM Trans. on Graph. **2**(2), 359–367 (1983)

16. Sumner, R.-W., O'Brien, J.-F., Hodgins, J.-K.: Animating sand, mud, and snow. Comput. Graph. Forum **18**(1), 17–26 (1999)
17. Thomas, K.-T.: Large scale studies of development of snowdrifts around buildings. J. Wind Eng. Indust. Aerodyn. **91**(6), 829–839 (2003)
18. Wei, X.-M., Zhao, Y., Fan, Z., Li, W., Yoakum, S., Kaufman, A.: Blowing in the wind. In: D. Breen, M. Lin (eds.) Proc. Eurographics/SIGGRAPH Symposium on Computer Animation'2003, pp. 75–85. Eurographics Association, San Diego, CA (2003)






CHANGBO WANG is a PhD candidate at the State Key Lab. of CAD & CG, Zhejiang University, People's Republic of China. He received his BE degree in 1998 and his ME degree in Civil Engineering in 2002, respectively, both from Wuhan University of Technology. His research interests include physically based modeling and rendering, computer animation and realistic image synthesis.

ZHANGYE WANG is an associate professor at the State Key Lab. of CAD & CG, Zhejiang University, People's Republic of China. He received his BSc degree in Physics in 1987 and his MSc degree in Optics in 1990, respectively, both from East China Normal University. In 2002, he received his PhD in Computer Graphics from Zhejiang University. His research interests include realistic image synthesis, computer animation and virtual reality.

TIAN XIA is currently a graduate student of computer science at University of Illinois, Urbana-Champaign, USA. He received his BE in Computer Science from Zhejiang University, People's Republic of China. His major research interest is realistic image synthesis.

QUNSHENG PENG is a professor of the State Key Lab. of CAD & CG at Zhejiang University. His research interests include realistic image synthesis, virtual reality, infrared image synthesis, point-based rendering, scientific visualization and biological calculation, etc. He graduated from Beijing Mechanical College in 1970 and received a PhD from the Department of Computing Studies, University of East Anglia, in 1983. He is currently the Vice Chairman of the Academic Committee, State Key Lab. of CAD & CG, Zhejiang University and is serving as a member of the editorial boards of several Chinese journals.